

# BIG DATA ANALYTICS & APPLICATIONS

## Lecture 02: The 6 Vs & Data Integration

Aifee Aadil

### What this lecture teaches:

1. The 6 Vs in depth — the problem each V causes and the tools that solve it.
2. How data gets into a Big Data system (ETL vs ELT, batch vs stream ingestion).
3. Kafka, and the Lambda and Kappa architectures.

## 1. Recap & Why RDBMS Breaks

### From Lecture 01

- Data grew from **GB** (RDBMS) to **PB/EB** (distributed systems).
- Scale-Up hits a limit; Scale-Out gives **elastic** capacity (grow or shrink easily).
- Schema-on-Write = strict, slow in. Schema-on-Read = flexible, fast in.
- Data Locality = send small code (KB) to big data (TB).

### 4 reasons RDBMS breaks with modern data

#### 1. Storage limit

One server's disk cannot hold multi-terabyte log tables.

#### 2. ACID lock overhead

Strict ACID needs **global locks** → only one writer at a time.

1000 users write → lock → 1 writes, 999 wait → very slow

#### 3. Fixed schema

Changing a column on a **10-billion-row** table can lock the database for **hours**.

#### 4. Network bottleneck

All data must pass through one **NIC** (Network Interface Card = the computer's network "door").

### Simple:

One disk, global locks, rigid tables, one network door → RDBMS can't keep up.

## 2. The 6 Vs Framework

In **2001**, **Doug Laney** (Gartner) described Big Data with **3 Vs**: Volume, Velocity, Variety.  
Today we use **6 Vs**.

| About the data itself         | About trust, links & money    |
|-------------------------------|-------------------------------|
| 1. Volume → how much          | 4. Veracity → how trustworthy |
| 2. Velocity → how fast        | 5. Valence → how connected    |
| 3. Variety → how many formats | 6. Value → how useful (ROI)   |

**Easy Memory Trick:**

First 3 (Volume, Velocity, Variety) → Laney 2001  
modern additions

Last 3 (Veracity, Valence, Value) →

### 3. V1: Volume — How Much

**Volume** = the total amount of data produced by apps, IoT sensors, clicks, and system logs.

#### When does size become a problem?

Terabyte ( $10^{12}$ ) → one strong server can still manage



Petabyte ( $10^{15}$ ) → one server's indexing, backup, restore fail



Exabyte ( $10^{18}$ ) → need many data centres in different countries

- **Indexing** → making a “table of contents” to find data fast.
- **Backup / Restore** → making a safety copy / getting data back from it.
- **Geo-replication** → copies kept in data centres in different places.

**Example:**

Backing up 1 PB at 1 GB/sec takes about **11.6 days** on one machine.

#### Problem 1: Small Files Problem

In Hadoop, the master node (**NameNode**) keeps **metadata** (file name, size, location) of **every file** in its **RAM**.

Millions of tiny files → millions of metadata entries → NameNode RAM full → cluster slows/crashes

**Example:**

100 million files of 1 KB = only 100 GB of data, but 100 million entries in RAM.  
The **number** of files hurts, not the size.

**Fix: compaction** = merge small files into big ones (**SequenceFile, Parquet**).

**Simple:**

Too many tiny files fill the master's memory. Merge them.

## Problem 2: Storage grows, CPU speed doesn't

Data grows very fast, but CPU clock speed has been almost flat. Reading data (**I/O**) is the slow part.

**Fix: compression (Snappy, ZSTD).**

Read 3 GB compressed + CPU unzips is faster than read 10 GB raw

### Simple:

Spend a little CPU to read much less data.

## Volume tools

| Tool                  | How it handles volume                        | Used for                  |
|-----------------------|----------------------------------------------|---------------------------|
| <b>Apache HDFS</b>    | Splits files into 128 MB blocks across nodes | On-premise Hadoop storage |
| <b>AWS S3 / GCS</b>   | Cloud object storage, "infinite" scaling     | Cloud data lakes          |
| <b>Apache Iceberg</b> | Metadata index for multi-PB tables           | Lakehouse table format    |
| <b>ClickHouse</b>     | Columnar compression (~10:1)                 | Fast analytics on PB logs |

- **On-premise** → on the company's own servers.
- **S3** = Amazon Simple Storage Service; **GCS** = Google Cloud Storage.
- **Lakehouse** → Data Lake + Data Warehouse features (tables, ACID, fast SQL).
- **10:1 compression** → 10 TB data takes about 1 TB disk.

## PySpark example

**PySpark** = using Apache Spark from Python.

```
spark = SparkSession.builder.appName("VolumeInspection").getOrCreate()
df = spark.read.parquet("s3a://bigdata-benchmark-bucket/web_logs/") # 500GB

print(df.count()) # total records
print(df.rdd.getNumPartitions()) # how many pieces

df_optimized = df.repartition(100, "country")
df_optimized.write.partitionBy("country").mode("overwrite") \
    .parquet("s3a://bigdata-benchmark-bucket/processed_logs/")
```

- **SparkSession** → opens a connection to the Spark cluster.
- **read.parquet("s3a://...")** → loads 500 GB from Amazon S3 into a **DataFrame** (distributed table).

- `count()` → every node counts its own part, then totals are added.
- `getNumPartitions()` → how many pieces the data is split into.
- `repartition(100, "country")` → reorganise into 100 pieces, grouped by country.
- `partitionBy("country")` → save one folder per country → a query for "BD" reads only the BD folder.

**Simple:**

Spark splits big data into partitions; organising them well makes later queries faster.

## 4. V2: Velocity – How Fast

**Velocity** = how fast data is created, taken in, and processed to get a useful result.

### The Latency Spectrum

**Latency** = waiting time until you get the result.

| Type             | Delay             | Starts by                 | Example                          |
|------------------|-------------------|---------------------------|----------------------------------|
| Batch            | Hours / days      | Scheduled <b>CRON</b> job | Month-end accounts               |
| Micro-batch      | Seconds / minutes | Time window               | Live dashboard (Spark Streaming) |
| Real-time stream | < 100 ms          | Each event                | Card fraud, stock trading        |

- **CRON job** → a task that runs at a fixed time (e.g. every night at 2 AM).
- **Time window** → collect data for a few seconds, then process that small group.

**Easy Memory Trick:**

Batch → **daily**      Micro-batch → **every few seconds**      Stream → **instantly**

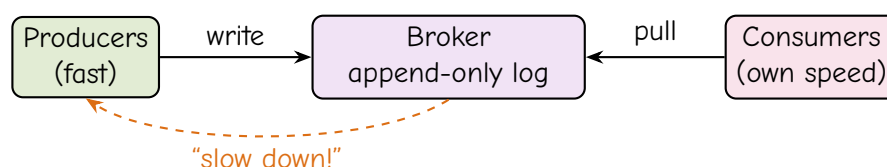
### Buffering & Backpressure

**Problem:** data comes faster than the system can save it → system crashes.

**Example:**

Eid sale: 1 lakh orders/sec arrive, database saves only 20,000/sec.

**Buffer** = a waiting area for data (like a water tank between a fast tap and a slow drain).



1. **Decoupling** → **producers** (senders) write to an **append-only log** at full speed. (Append-only = only add at the end, never change old data — like a diary.)

2. **Pull model** → **consumers** (processors) take data at their own speed.
3. **Backpressure** → if the **broker's** memory fills up, it tells producers to slow down.

**Simple:**

A buffer sits between fast senders and slow receivers so nothing crashes.

## Event-Time vs Processing-Time

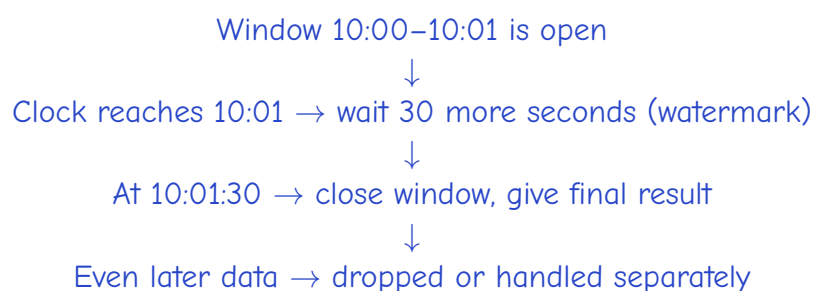
Fast data often arrives **out of order** (slow network, phone offline).

- **Event-Time** → when it **actually happened** on the device. (User clicked at 10:00:01.)
- **Processing-Time** → when the **server processed** it. (Arrived at 10:05:22.)

**Example:**

You order food on a bus with no signal at 10:00. The order reaches the server at 10:05. If we count by processing-time, it falls in the wrong minute.

**Watermark** = how long to wait for late data before closing a time window.



**Used in:** Apache Flink, Spark Streaming

**Simple:**

Event-time = when it happened. Processing-time = when the server saw it. Watermark = how long to wait for latecomers.

## Velocity tools

| Tool                   | How it works                                  | Delay           |
|------------------------|-----------------------------------------------|-----------------|
| <b>Apache Kafka</b>    | Distributed commit log, fast partition writes | 2–10 ms         |
| <b>Apache Flink</b>    | Stateful, true event-by-event streaming       | Sub-millisecond |
| <b>Spark Streaming</b> | Micro-batches                                 | 100 ms – 1 s    |
| <b>AWS Kinesis</b>     | Managed cloud streaming service               | 200–500 ms      |

- **Commit log** → ordered, append-only list of events.
- **Stateful** → remembers past data (e.g. last 10 payments of each card).
- **Managed service** → the cloud company runs the servers for you.

## Example: IoT message in Kafka

```
{
  "event_id": "EVT-2026-991823",
  "event_timestamp": 1789123400000,
  "device_id": "SENS-RANGPUR-092",
  "readings": { "temperature": 38.4, "humidity": 79.2,
                "vibration_index": 0.04 },
  "metadata": { "kafka_topic": "iot_telemetry_live",
                "partition": 2, "offset": 883910 }
}
```

- `event_id` → unique ID (used to remove duplicates).
- `event_timestamp` → **event-time** in milliseconds since 1 Jan 1970.
- `readings` → the sensor values.
- `topic`, `partition`, `offset` → where the message sits inside Kafka (see the Kafka section).

## 5. V3: Variety — How Many Formats

**Variety** = how different the data formats are.

- **Structured (10–15%)** → fixed tables (RDBMS, CSV).
- **Semi-structured (15–20%)** → labels inside the data (JSON, XML, YAML, NoSQL documents).
- **Unstructured (70–80%)** → no structure (images, MP4, PDF, audio).

**CSV** = Comma-Separated Values → text file, one row per line: 1001,Aadil,CSE.

### Text vs Binary formats

- **Text** (CSV, JSON) → humans can read it, but it is big and slow to read.
- **Binary** (Parquet, Avro, ORC) → machine format, small and fast.

### File format comparison

| Format  | Layout         | Schema change | Splittable? | Best for          |
|---------|----------------|---------------|-------------|-------------------|
| CSV     | Row, text      | Poor          | Sometimes   | Quick exports     |
| JSON    | Row, text      | Medium        | Hard        | Web APIs          |
| Parquet | Column, binary | High          | Yes         | Analytics (Spark) |
| Avro    | Row, binary    | Excellent     | Yes         | Kafka streaming   |
| ORC     | Column, binary | High          | Yes         | Hive / Hadoop     |

- **Schema evolution** → can you add/remove columns later without breaking old data?

- **Splittable** → can one big file be cut so **many machines read it in parallel**? If not, one machine must read the whole file alone.
- **Schema Registry** → a central place that stores all versions of Avro schemas.
- **ORC** = Optimized Row Columnar.

## Row vs Column storage

| ID | Name   | Salary |
|----|--------|--------|
| 1  | Aadil  | 50000  |
| 2  | Nabila | 60000  |

### 1. Row-oriented (CSV, Avro, RDBMS)

[1, Aadil, 50000] [2, Nabila, 60000]

- Very fast to **write** one record.
- To read 1 column, it must read **whole rows**.

### 2. Column-oriented (Parquet, ORC)

ID: [1, 2]    Name: [Aadil, Nabila]    Salary: [50000, 60000]

- Reads **only the needed columns**.
- Compresses very well (>80%) — same-type values sit together.
- **Vectorized reads** → CPU processes many values of one column at once.
- Slow to update a single row.

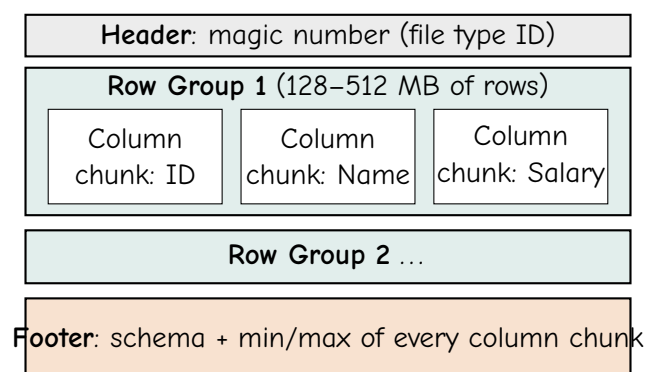
#### Example:

SELECT AVG(Salary) → row format reads ID + Name + Salary; column format reads only Salary.

#### Easy Memory Trick:

Row → like a **register book** (good for writing)      Column → like **separate lists** (good for analysing)

## Inside a Parquet file



- **Header** → a magic number that says “this is a Parquet file”.
- **Row group** → a big horizontal chunk of rows.
- **Column chunk** → one column’s data inside a row group.
- **Footer** → schema + **min/max** values of every column chunk (most important part).

### Predicate Pushdown

Spark reads the **footer first** and skips chunks that cannot match.  
(**Predicate** = the WHERE condition.)

Query: WHERE salary > 100000  
 ↓  
 Footer says Row Group 1: max salary = 80000  
 ↓  
 No row can match → **skip the whole row group**

#### Simple:

Parquet keeps a summary in the footer, so Spark skips data without reading it.

### Which format where?

| Layer               | Format                 | Why                             |
|---------------------|------------------------|---------------------------------|
| Streaming input     | Avro                   | Fast, checks schema exactly     |
| Raw data lake       | JSON / compressed text | Keeps original data unchanged   |
| Analytics warehouse | Parquet / Delta Lake   | Fast column reading             |
| ML feature store    | Parquet                | Fast reading of feature columns |

- **Serialization** → turning data into bytes to send or store.
- **Delta Lake** → Parquet + ACID transactions.
- **Feature store** → storage for ready-made ML input values.

#### Easy Memory Trick:

Writing in (stream) → **Avro** (row)      Reading out (analytics) → **Parquet** (column)

### Parquet vs CSV in code

```
# CSV: must read every row and parse text
df_csv = spark.read.option("header", "true").csv("s3a://my-bucket/logs.csv")
df_csv.filter("status = 'ERROR']").select("user_id").count()      # slow

# Parquet: reads only status and user_id columns
df_parquet = spark.read.parquet("s3a://my-bucket/logs.parquet")
df_parquet.filter("status = 'ERROR']").select("user_id").count()  # fast
```

Result: Parquet is often **10–100× faster** and reads **90% less data**.

## 6. V4: Veracity — How Trustworthy

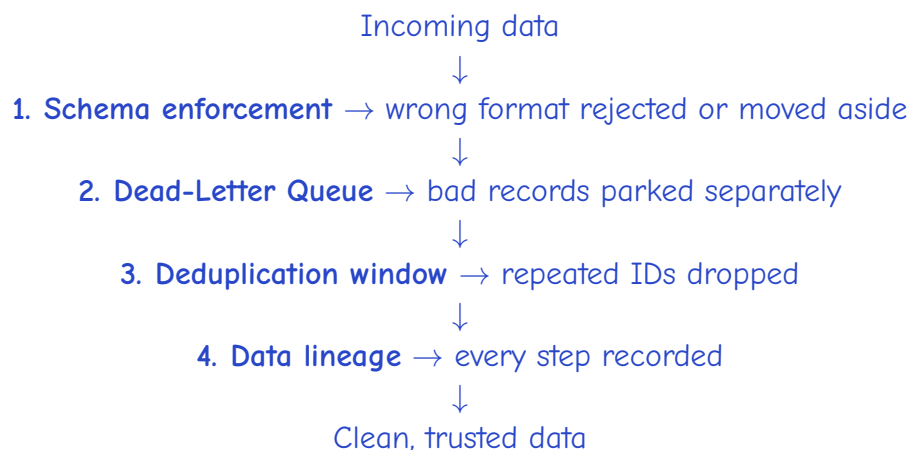
**Veracity** = how much we can trust the data.

Problems: **uncertainty**, **noise** (random wrong values), **bias** (one-sided data), **duplicates**, **corrupted** messages.

### What causes bad data?

1. **Broken sensors** → impossible values like **9999°C**.
2. **Network timeouts** → Kafka producer resends → same message twice (**duplicate**).
3. **Upstream API changes** → source renames "temp" to "temperature" → pipeline breaks.
4. **Clock drift** → device clocks slowly go wrong → events in wrong order.

### How to keep data clean



- **Dead-Letter Queue (DLQ)** → a separate queue for bad records, so the pipeline **doesn't** crash.
- **Deduplication window** → remember IDs seen in the last few minutes; drop repeats.
- **Data lineage (provenance)** → full history of where data came from and how it changed.

### Example:

DLQ is like the post office's "undeliverable letters" box — other letters keep moving.

## Veracity tools

| Tool                 | What it does                                           |
|----------------------|--------------------------------------------------------|
| Great Expectations   | Automatic data tests (e.g. “column X cannot be empty”) |
| Delta Lake / Iceberg | ACID + schema rules → no half-written broken files     |
| Apache OpenLineage   | Tracks and draws data lineage                          |
| Debezium (CDC)       | Copies every database change exactly to other systems  |

**CDC** = Change Data Capture → watch a database’s change log and send every insert/update/delete as an event.

### Simple:

Check data, park bad records, remove duplicates, and record its history.

## 7. V5: Valence — How Connected

**Valence** = how much data items are **linked** to each other.

- **Entity** → a thing (person, account, product).
- **Graph** → entities = **nodes** (dots), links = **edges** (lines).

### Why connections make work harder

Two datasets, both **1 TB**:

- **A** = separate log lines → easy, split and process in parallel.
- **B** = a tightly connected graph → much harder (must follow links across the data).

**Hop** = following one link. “Friends of friends of friends” = 3 hops.

$$1 \text{ hop: } N \quad 2 \text{ hops: } N^2 \quad 3 \text{ hops: } N^3 \quad k \text{ hops: } O(N^k)$$

### Example:

200 friends each → 3 hops =  $200^3 = 8 \text{ million}$  paths. This is **combinatorial explosion**.

## SQL JOIN vs Graph engine

### 1. SQL (RDBMS)

3 hops = **3 huge JOINS** over whole tables → lots of RAM, CPU, disk.

### 2. Graph engine (pointer chasing)

Each node directly stores links (**pointers**) to its neighbours.

It just follows links from  $A \rightarrow B \rightarrow C$ .

Time depends only on the **part of the graph visited**, not total data size.

**Example:**

SQL = search the whole phone book every time.

Graph = each friend already has their friends' numbers saved.

| Tool             | Type                        | Used for                          |
|------------------|-----------------------------|-----------------------------------|
| Neo4j / Memgraph | Graph database              | Fraud rings, recommendations      |
| Spark GraphX     | Distributed graph computing | PageRank, big graph analytics     |
| AWS Neptune      | Cloud graph DB              | Social networks, knowledge graphs |

**PageRank** → Google's method of ranking pages by links. **Fraud ring** → a group of linked fake accounts.

**Simple:**

Highly connected data needs graph engines; SQL JOINS explode.

## 8. V6: Value — How Useful

**Value** = the money, savings, or advantage we get from data.

**ROI** = Return on Investment → how much we gain compared to what we spend.

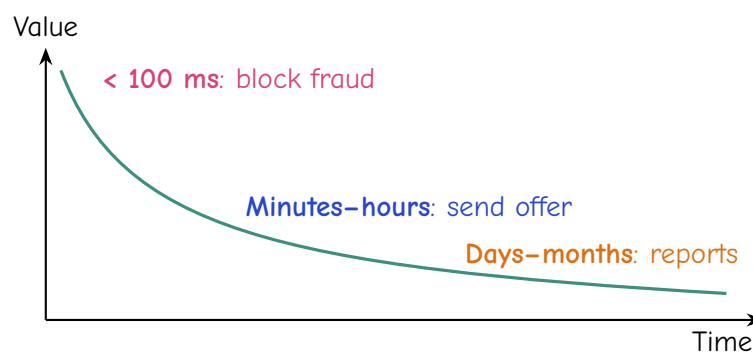
### The Big Data Paradox

- **One record** → almost worthless.
- **Millions together** → very valuable (ML models, hidden trends).

**Example:**

One click on a product = nothing. 100 million clicks = you know what customers want.

### Value decays with time



| Time          | Value type            | Action                              |
|---------------|-----------------------|-------------------------------------|
| < 100 ms      | Operational (highest) | Block a fraud card payment          |
| Minutes–hours | Tactical              | Send an offer to an active app user |
| Days–months   | Strategic             | Quarterly sales forecast            |

## Value tools

- **Databricks / Spark MLlib** → machine learning and predictions.
- **BigQuery / Snowflake** → **serverless** warehouses (no servers to manage) for BI queries.
- **DuckDB** → small, fast analytics database inside your own program.
- **Superset / Tableau** → dashboards and charts.

### Simple:

One record is worthless; millions are valuable — if you act fast.

## 9. 6 Vs Summary

| V               | Question       | Measured by              | Strategy                          | Tools                           |
|-----------------|----------------|--------------------------|-----------------------------------|---------------------------------|
| <b>Volume</b>   | How much?      | TB, PB, EB               | Distributed storage, partitioning | HDFS, S3, Iceberg               |
| <b>Velocity</b> | How fast?      | Events/sec, ms           | Buffering, watermarks, streaming  | Kafka, Flink, Spark Streaming   |
| <b>Variety</b>  | What formats?  | Struct., semi, unstruct. | Schema-on-Read, columnar          | Parquet, Avro, JSON             |
| <b>Veracity</b> | How true?      | Error %, duplicate %     | Quality rules, DLQ                | Great Expectations, Delta Lake  |
| <b>Valence</b>  | How connected? | Links per node           | Graph engines                     | GraphX, Neo4j, Neptune          |
| <b>Value</b>    | What benefit?  | Revenue, fraud saved     | ML, scalable SQL                  | Databricks, BigQuery, Snowflake |

## Which V is it?

| Scenario                                         | V               |
|--------------------------------------------------|-----------------|
| 10 years of world satellite images               | <b>Volume</b>   |
| Check card swipes in under 50 ms                 | <b>Velocity</b> |
| Web logs + JSON + MP3 call recordings together   | <b>Variety</b>  |
| Remove wrong readings from faulty sensors        | <b>Veracity</b> |
| Trace money laundering across 50 shell companies | <b>Valence</b>  |

**Easy Memory Trick:**

Look for the key word: **size** → Volume, **ms** → Velocity, **many formats** → Variety, **faulty/wrong** → Veracity, **paths/links** → Valence, **profit/benefit** → Value

## 10. ETL vs ELT

**Data integration** = bringing data from many sources into one place.

### 1. ETL (Extract → Transform → Load)

Old data warehouse way (1990s–2000s).

**Extract from OLTP DB → Transform on a central ETL server → Load into Warehouse**

- Clean and reshape **before** storing.
- Tools: **Informatica, Talend, SSIS** (SQL Server Integration Services).

#### Why ETL fails at Big Data scale

1. **One ETL server** → becomes the bottleneck.
2. **Data lost forever** → unused fields are removed during cleaning.
3. **Breaks easily** → a new column upstream breaks the job.
4. **No unstructured support** → can't handle JSON, images, video well.

#### Example:

In 2022 you removed “device type”. In 2025 the boss asks “mobile vs desktop sales since 2022?” → impossible.

#### Simple:

Clean first, then store.

**Used in:** Traditional data warehouses

### 2. ELT (Extract → Load → Transform)

Big Data way — the order is **flipped**.

**Extract from anywhere → Load raw into Data Lake → Transform later inside the cluster**

- Raw data stored **immediately, unchanged**.
- Transform **when needed**, using Spark, SQL, or **dbt**.
- Raw data kept forever → can re-transform anytime.

#### Simple:

Store raw first, clean later.

**Used in:** Data lakes, cloud warehouses

**Easy Memory Trick:**

**ETL** = T before L = Schema-on-**Write**

**ELT** = L before T = Schema-on-**Read**

## ETL vs ELT

|                        | ETL                       | ELT                               |
|------------------------|---------------------------|-----------------------------------|
| <b>Transform where</b> | One central ETL server    | Big distributed cluster           |
| <b>Load when</b>       | After cleaning            | Immediately                       |
| <b>Raw data</b>        | Thrown away               | Kept forever                      |
| <b>Scaling</b>         | Limited by one server     | Grows with cluster (linear)       |
| <b>Flexibility</b>     | Low (changes take weeks)  | High (schema-on-read)             |
| <b>Tools</b>           | Informatica, Talend, SSIS | Spark, dbt, Databricks, Snowflake |

## Modern ELT tool stack



- **dbt** = data build tool → write SQL, it builds tables in the right order.
- **Orchestration** → decides what runs when (“transform only after load finishes”). Like an orchestra conductor.

## 11. Batch vs Stream Ingestion

**Ingestion** = bringing data into the system.

### 1. Batch Ingestion

Move data at **fixed times** (nightly, hourly).

**Collect all day → copy in bulk at 1 AM**

- Moves huge amounts efficiently (high throughput).

**Used in:** Apache **Sqoop** (SQL → Hadoop), AWS DataSync, scripts

## 2. Stream Ingestion

Move each event **as it happens**.

Card swipe → Kafka in milliseconds

- Low latency, real-time.

**Used in:** Apache Kafka, Debezium (CDC), AWS Kinesis

### Easy Memory Trick:

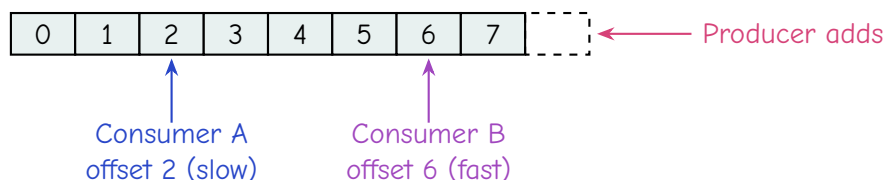
Batch → **bus** (waits, carries many)

Stream → **motorbike** (goes right away)

## 12. Apache Kafka

Kafka = a **distributed append-only log** for streaming data.

- **Broker** → one Kafka server.
- **Topic** → a named channel (e.g. payments).
- **Partition** → a topic is split into parts on different brokers (parallel).
- **Producer** → sends messages; always adds at the **end** (tail).
- **Consumer** → reads messages.
- **Offset** → position number of a message (0, 1, 2, ...).



**Key benefit:** each consumer keeps its **own offset** and **pulls** at its own speed. Fast producers **never crash** slow consumers — data just waits in the log.

### Example:

Like a YouTube video: each viewer watches at their own position and can rewind.

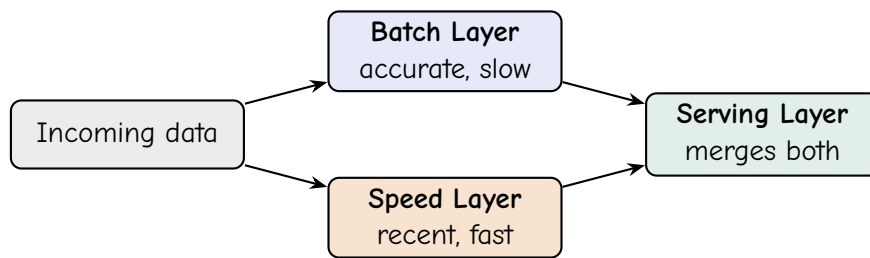
### Simple:

Kafka = a shared diary. Writers add at the end; each reader keeps their own bookmark.

## 13. Lambda vs Kappa Architecture

### 1. Lambda Architecture (Nathan Marz)

Gets both **accurate history** and **real-time speed**, using 3 layers.



1. **Batch layer** → keeps all raw data (never changed) and computes exact results from time to time (Spark/Hadoop).
2. **Speed layer** → handles the newest data quickly, while the batch job is still running.
3. **Serving layer** → combines both to answer queries.

### Example:

Today's total sales = batch result (till midnight) + speed layer (since midnight).

### Problems with Lambda

- **Two codebases** → same logic written twice (Spark for batch, Flink/Storm for stream).
- **Sync bugs** → batch and stream results don't match exactly; hard to debug.

## 2. Kappa Architecture (Jay Kreps)

Removes the batch layer. **Everything is a stream.**

**Data → Kafka log (kept long) → One stream processor → Serving**

- Keep raw events in Kafka for a long time (**retention**).
- Need to recompute history? Start a second stream job and **replay from offset 0**.

### Simple:

Lambda = two pipelines + merge. Kappa = one stream pipeline; replay when needed.

## Lambda vs Kappa

|                          | Lambda                    | Kappa                   |
|--------------------------|---------------------------|-------------------------|
| <b>Layers</b>            | 3 (batch, speed, serving) | 2 (stream log, serving) |
| <b>Code</b>              | Two codebases             | One codebase            |
| <b>Processing</b>        | Batch + stream            | Stream only             |
| <b>Recompute history</b> | Scheduled batch jobs      | Replay the log          |
| <b>Maintenance</b>       | High                      | Low                     |
| <b>By</b>                | Nathan Marz               | Jay Kreps               |

### Easy Memory Trick:

Lambda → Lots of layers, two codes

Kappa → Keep it simple, one stream

## 14. Quick Revision

### Remember

- 6 Vs: Volume, Velocity, Variety (Laney 2001) + Veracity, Valence, Value.
- Volume: small files problem → compaction; compression trades CPU for less I/O.
- Velocity: batch / micro-batch / stream; buffer + backpressure; watermark for late data.
- Variety: row (write fast) vs column (read fast); Parquet footer → predicate pushdown.
- Veracity: schema checks, DLQ, deduplication, lineage.
- Valence: links make JOINS explode → graph engines.
- Value: one record worthless, millions valuable; value drops with time.
- ETL = clean then store. ELT = store raw then clean (Big Data way).
- Kafka: topic → partitions; producers append; consumers keep own offset.
- Lambda = batch + speed + serving. Kappa = stream only, replay log.

## 15. Exam Questions

**Q1. Why is Parquet much faster than CSV for `SELECT AVG(salary) FROM employee`?**

**Ans:** Parquet stores data by column, so it reads only the salary column; CSV must read every full row. Parquet is binary (no text parsing), compresses very well (less disk/network reading), and its footer tells where each column chunk is and its min/max values. Result: often 10–100× faster.

**Q2. A bank app tracks clicks to catch fraud in real time and wants to keep raw logs forever. ETL or ELT?**

**Ans: ELT.** ELT loads raw data unchanged, so logs are kept forever (ETL throws unused fields away). Clicks arrive fast; ELT with stream ingestion (Kafka) loads them instantly, while ETL's single server would slow it down. Fraud patterns change, and Schema-on-Read lets the team extract new features from old raw data later.

**Q3. What main headache does Kappa solve compared to Lambda?**

**Ans:** The **two codebases**. Lambda needs the same logic written twice (batch + stream), and merging both results causes sync bugs. Kappa uses one stream codebase and replays the Kafka log from offset 0 to recompute history.

### Practice Questions

1. Why does RDBMS break with modern data? Give 4 reasons.
2. Who gave the first 3 Vs? Write all 6 Vs with examples.
3. What is the small files problem? How is it fixed?
4. Explain batch, micro-batch, and real-time stream.
5. What are buffering and backpressure?
6. Event-time vs processing-time. What is a watermark?

7. Compare row and column storage.
8. Describe a Parquet file. What is predicate pushdown?
9. What does “splittable” mean?
10. What causes low veracity? Explain DLQ and data lineage.
11. Why do multi-hop JOINS explode? How do graph engines help?
12. Explain the value-decay curve.
13. Compare ETL and ELT.
14. Explain Kafka topics, partitions, producers, consumers, and offsets.
15. Compare Lambda and Kappa architectures.